

01-march-2026

Zomato = Outsourcing

Date _____
Page _____

Modern Async Patterns

- Promises and queue ye javascript ko javascript banati hai.
- one more topic is closure

21. Closure

- closure me samjhana jada chota jata hai, closure me mental model or kaise code execute hora, kaha per memory. jiski chiz jada important hai.

A closure is the combination of a function bundled together (enclosed) with references to its surrounding state (the lexical environment). In other words, a closure gives a function access to its outer scope.

ex.

```
function init() {  
  let name = "Mozilla";  
  function displayName() {  
    console.log(name);  
  }  
  displayName();  
}
```

function created
name ke pass he access inner function ka
name is local variable created by init
It is inner function that forms a closure
name variable declared in the parent function
execute kardo wahin ke wahin
init(); → function call

output ⇒ "Mozilla"

- name space dhundna, sabse pehle inner function apne pass dekhega nahi mila to outer ke pass dekhega, fir nahi mila to bahar jayega ise scope chaining bolte hai

Date _____
Page _____

whenever function created
usko ek chhota sa scope
miltta hai

- agar function ke andar variable declared hua vo bhi execution ke bad kham ho jata hai
ex, let name = "muku"; as long as scope khatm ho

```
function makefunc() {
```

function displayName()

3

```
return displayName;
```

3

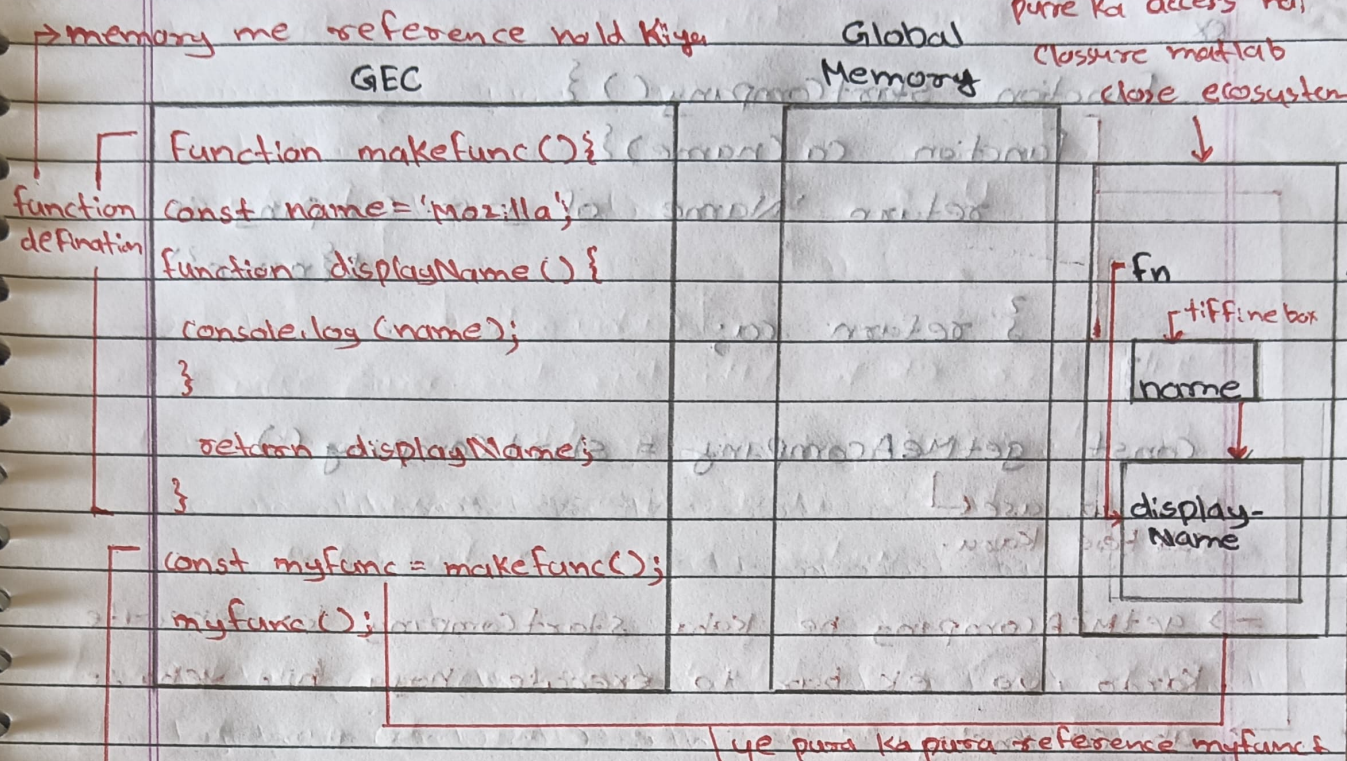
```
const myfunc = makefunc();
```

myfunc();

output $\Rightarrow$ Mozilla.

- function has its own scope called function scope.
- myfunc ke pass sirf function displayName ka access hai, par fir bhi uske pass name ka access hai, isse hi noscope khte hai, displayName upne sat reference leke aata hai

Note! Closure ko globally access nahi kar sakte hai.
 tiffine box ka ham `[[SCOPE]]` ← sirf symbol sehi kar sakte hai.
 private space of language itself, you can't access
 myfunc ke pass pure ka access hai
 closure matlab close ecosystem



→ makefunc() at least ek bar chalega or execute bhi hoga

- makefunc() ke value di ek function, uske bad uska scope delete ho jayega
- fir jo function mila tha vo myfunc() ke pass chala gya usne ab function ka reference hold kar liya
- or myfunc() ke under displayName return ho raha hai.
- Javascript ke pass special property hoti hai or ye tabhi execute hoti hai jab ap kisi function ke under note ho.

In JavaScript, closure are created every time a function is created, at function creation time.

- to ap jitne bhi function banate ho, jese makefunc uske sat ek closure hamega, displayName banaya uske bhi sat ek closure banega
- closure ek tiffine box hai, or har ek function ke sat banata hai, uska ka reference leke aata hai

ex.
function startCompany() {

function ca(name) {
return 'Name of your company is
\$\${name}\$';
}
return ca;

const getMeAcompany = startCompany();

How to use
call kare karu.

⇒ getMeAcompany ne kaha startCompany ko execute
kardo, vo ek bar to execute hoga hi, yeha se
ab ca melega

to getMeAcompany ke under ca function ka refera-
ce rakha hai defination rakha hai
to ca function ko jab call karte ho input vo
ca nahi hai uska reference ab getMeAcompany
ke pass hai.

const myCompanyName = getMeAcompany("Zomato")

- getMeAcompany ka reference myCompanyName
ke under rakha hua hai
is tarah closure use kiya jata hai.

- Or ye ek high order function bhik hai.

output: Name of your company is Zomato

Name of your company is Zomato

Note! - Sabse pass memory hoti hai or apna apna closure

Date _____
Page _____

```
ex,
function eternal (guest) {
  const guestName = guest;
  let count = 0;
  function zomato () {
    console.log('Hi, ${guestName}, from zomato');
  }
  function blinkit () {
    if (count === 1) return;
    console.log('Hi, ${guestName}, from blinkit');
    count++;
  }
  return {
    zomato,
    blinkit,
  };
}
```

one time return

function ka reference

imeditaly return nahi karke ho

```
const hitesh = eternal('hitesh');
const Piyush = eternal('piyush');
```

hitesh.blinkit() independent hai Piyush.blinkit se, closure ke karon

hitesh.blinkit();

hitesh.blinkit();

hitesh.blinkit();

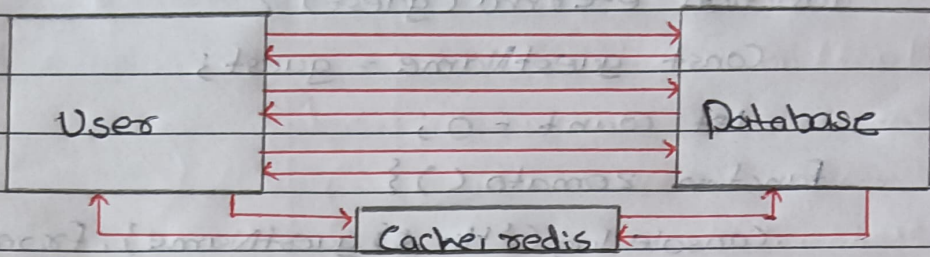
Output

Hi, hitesh, from blinkit

- React useMemo() hook, agar apke function me kuchh change nahi hua hai, to us function ko reference me hold karke rakhta hai, optimisation ke liye.

Note: memory fast
Harddisk slow

user database pe data ke liye request bhejta huwa



- Tab user ek hi data bar bar mang raha ho to,
- or database me agar aap filese read karke ki memory karke isse expensive hoga.

~~memory~~ memory ke ander, kyu ki vo fast hoti hai.

- cache or redis data ko memory me rakh te hai,
- ram ke ander user ko sidha iske pass hi miljata hai data barbar database ke pass nahi jana hata fir.

ex.

```
Const cups = ["green", "blue", "red"]  
cups.map
```

- map apne aap ek ek value pe jayega
- map ke pass bhi khadi ek memory hoti closure jesi.
- closure function ke run time ke toh dooran banta hai, ye har function ke sat hote hai,
- its static with them isliye lexical bolte hai,
- JavaScript lexical scoping language hai.